

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Деревья решений»

Выполнил:

Студент 4 курса

Группы ИИ-25

Заседатель Н.С.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: на практике сравнить работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.

Задание:

1. Загрузить датасет по варианту.
2. Разделить данные на обучающую и тестовую выборки.
3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost).
4. Оценить точность каждой модели на тестовой выборке.
5. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 6

- Pima Indians Diabetes
- Предсказать наличие диабета у пациенток
- **Задания:**
 1. Загрузите данные и стандартизируйте их.
 2. Разделите выборку в соотношении 70/30.
 3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost).
 4. Сравните производительность моделей с помощью метрики recall для положительного класса (наличие диабета).
 5. Обоснуйте, какой из алгоритмов предпочтительнее для данной медицинской задачи.

Ход работы:

0. Общие особенности

- язык и библиотеки: Python, scikit-learn (дерево решений, случайный лес, AdaBoost), XGBoost, CatBoost;
- разбиение: 70 % обучающая, 30 % тестовая выборка, со стратификацией по классам (доля больных одинакова в обеих частях);
- стандартизация (StandardScaler) обучается только по обучающей выборке и применяется к тестовой, чтобы информация о тесте не попадала в обучение. Для деревьев масштаб признаков на результат не влияет, стандартизация выполнена по требованию задания;
- гиперпараметры заданы заранее и по тестовой выборке не подбирались;
- задание было выполнено в двух режимах: со стандартными настройками и с учётом дисбаланса классов (в данных 65 % здоровых и 35 % больных);
- помимо одного разбиения 70/30 (random_state = 42), также повторили на 20 случайных разбиениях 70/30 для оценки устойчивости (среднее $\pm$ стандартное отклонение).

1. Загрузка датасета

Набор Pima Indians Diabetes содержит 768 пациенток и 8 числовых признаков:

Признак	Описание
Pregnancies	число беременностей
Glucose	концентрация глюкозы в плазме через 2 часа после перорального теста на толерантность к глюкозе
BloodPressure	диастолическое давление, мм рт. ст.
SkinThickness	толщина кожной складки трицепса, мм
Insulin	инсулин сыворотки через 2 часа, мкЕд/мл
BMI	индекс массы тела
DiabetesPedigree	функция наследственной предрасположенности к диабету
Age	возраст, лет

Целевая переменная Outcome: 1 (диабет) у 268 пациенток (34.9 %), 0 (нет диабета) у 500. Пропусков в явном виде нет, дубликатов нет. Первые строки файла содержат комментарии с описанием признаков, они пропускаются при чтении.

Замечание: в признаках Glucose, BloodPressure, SkinThickness, Insulin, BMI нулевые значения физиологически невозможны и фактически означают пропуски (например, Insulin = 0 у 374 пациенток). В работе они не заменялись, чтобы не выходить за рамки задания.

2. Разделение на обучающую и тестовую выборки

Разбиение 70/30 со стратификацией:

Выборка	Объектов	Из них больных
Обучающая	537	187
Тестовая	231	81

Тестовая выборка небольшая, поэтому одного разбиения недостаточно для надёжных выводов, и дополнительно использованы 20 повторных разбиений.

3. Обучение моделей

Модель	Основные параметры
Одиночное дерево	max_depth = 4, min_samples_leaf = 5
Случайный лес	300 деревьев, min_samples_leaf = 3
AdaBoost	200 деревьев - «пней» (max_depth = 1), learning_rate = 0.5
XGBoost	200 деревьев, max_depth = 3, learning_rate = 0.05, subsample = 0.8, colsample_bytree = 0.8
CatBoost	300 итераций, depth = 4, learning_rate = 0.05

Ограничение глубины одиночного дерева нужно для снижения переобучения.

Режим с учётом дисбаланса классов: class_weight='balanced' для дерева, леса и базовых деревьев AdaBoost; scale_pos_weight (отношение числа здоровых к больным в обучающей выборке) для XGBoost; auto_class_weights='Balanced' для CatBoost.

4. Оценка моделей на тестовой выборке(результат работы программы)

Метрики: Recall для класса «диабет» (основная), Precision, F1, Accuracy, ROC-AUC. Матрицы ошибок записаны как [[TN, FP], [FN, TP]]. Recall = TP / (TP + FN) показывает долю найденных больных.

Данные: (768, 8), диабет: 268 (34.9%), нет диабета: 500
Стандартные настройки (без весов классов)

Одно разбиение 70/30 (random_state=42)

Обучающая: 537 (диабет: 187), тестовая: 231 (диабет: 81)

	Recall	Precision	F1	Accuracy	ROC-AUC
Дерево решений	0.5926	0.6857	0.6358	0.7619	0.7901
Случайный лес	0.5556	0.6818	0.6122	0.7532	0.8278
AdaBoost	0.5185	0.7000	0.5957	0.7532	0.8340
XGBoost	0.5556	0.6818	0.6122	0.7532	0.8318
CatBoost	0.5062	0.6406	0.5655	0.7273	0.8304

Матрицы ошибок [[TN, FP], [FN, TP]]:

Дерево решений	TN=128	FP= 22	FN= 33	TP= 48
Случайный лес	TN=129	FP= 21	FN= 36	TP= 45
AdaBoost	TN=132	FP= 18	FN= 39	TP= 42
XGBoost	TN=129	FP= 21	FN= 36	TP= 45
CatBoost	TN=127	FP= 23	FN= 40	TP= 41

Проверка устойчивости

Среднее ± std по 20 случайным разбиениям 70/30:

	Recall	Precision	F1	Accuracy	ROC-AUC
Дерево решений	0.566 ± 0.094	0.635 ± 0.045	0.592 ± 0.054	0.731 ± 0.023	0.774 ± 0.023
Случайный лес	0.565 ± 0.048	0.702 ± 0.043	0.625 ± 0.039	0.763 ± 0.023	0.826 ± 0.019
AdaBoost	0.543 ± 0.056	0.683 ± 0.045	0.604 ± 0.045	0.751 ± 0.024	0.822 ± 0.019
XGBoost	0.582 ± 0.042	0.673 ± 0.039	0.623 ± 0.034	0.754 ± 0.022	0.816 ± 0.019
CatBoost	0.575 ± 0.046	0.668 ± 0.043	0.617 ± 0.038	0.750 ± 0.024	0.822 ± 0.020

С учётом дисбаланса классов (balanced)

Одно разбиение 70/30 (random_state=42)

Обучающая: 537 (диабет: 187), тестовая: 231 (диабет: 81)

	Recall	Precision	F1	Accuracy	ROC-AUC
Дерево решений	0.6173	0.6849	0.6494	0.7662	0.7901
Случайный лес	0.5802	0.6351	0.6065	0.7359	0.8392
AdaBoost	0.5926	0.6154	0.6038	0.7273	0.8088
XGBoost	0.6667	0.6279	0.6467	0.7446	0.8304
CatBoost	0.5926	0.6234	0.6076	0.7316	0.8279

Матрицы ошибок [[TN, FP], [FN, TP]]:

Дерево решений	TN=127	FP= 23	FN= 31	TP= 50
Случайный лес	TN=123	FP= 27	FN= 34	TP= 47
AdaBoost	TN=120	FP= 30	FN= 33	TP= 48
XGBoost	TN=118	FP= 32	FN= 27	TP= 54
CatBoost	TN=121	FP= 29	FN= 33	TP= 48

Проверка устойчивости

Среднее ± std по 20 случайным разбиениям 70/30:

	Recall	Precision	F1	Accuracy	ROC-AUC
Дерево решений	0.747 ± 0.072	0.575 ± 0.047	0.646 ± 0.031	0.713 ± 0.033	0.781 ± 0.029
Случайный лес	0.652 ± 0.046	0.658 ± 0.031	0.654 ± 0.030	0.759 ± 0.020	0.825 ± 0.020
AdaBoost	0.627 ± 0.065	0.633 ± 0.048	0.627 ± 0.035	0.739 ± 0.027	0.800 ± 0.022
XGBoost	0.704 ± 0.053	0.621 ± 0.037	0.659 ± 0.037	0.745 ± 0.028	0.817 ± 0.019
CatBoost	0.693 ± 0.050	0.631 ± 0.040	0.659 ± 0.037	0.749 ± 0.028	0.819 ± 0.020

5. Сравнение результатов и выводы

5.1. Общие наблюдения

1. Результаты одного разбиения ненадёжны. Тестовая выборка содержит всего 81 больную: один пропущенный больной снижает recall примерно на 1.2 %. В стандартном режиме на одном разбиении лучший recall у одиночного дерева (0.593), а у CatBoost худший (0.506), но при усреднении по 20 разбиениям recall всех моделей лежит в диапазоне 0.54 – 0.58 при разбросе 0.04 – 0.09, поэтому различия между ними не отличимы от случайных колебаний.
2. Учёт дисбаланса классов существенно повышает recall. Средний recall вырос: у дерева с 0.566 до 0.747 (+0.181), у случайного леса с 0.565 до 0.652 (+0.087), у AdaBoost с 0.543 до 0.627 (+0.084), у XGBoost с 0.582 до 0.704 (+0.122), у CatBoost с 0.575 до 0.693 (+0.118). Платой стало небольшое снижение precision и accuracy. Для данной задачи выбор весов классов влияет на recall сильнее, чем выбор алгоритма.
3. Высокий recall одиночного дерева не означает лучшего качества. С весами классов дерево имеет наибольший recall (0.747), но и наименьшую precision (0.575), наименьший ROC-AUC (0.781) и наибольший разброс (± 0.072). Это свидетельствует скорее о том, что дерево чаще относит пациенток к больным, а не о лучшем разделении классов. Ансамбли имеют ROC-AUC 0.80–0.83, то есть разделяют классы заметно лучше.
4. Ансамбли стабильнее одиночного дерева (меньший разброс метрик между разбиениями).

5.2. Применимость методов

- **одиночное дерево.** Преимущество: простота и интерпретируемость (правила видны врачу). Недостатки: низкое ROC-AUC, наибольшая нестабильность, склонность к переобучению. Подходит как интерпретируемая базовая модель, но не как основной классификатор;
- **случайный лес.** Наибольшая precision и ROC-AUC (0.826 в стандартном режиме, 0.825 с весами), наименьшая доля ложных тревог среди ансамблей, но recall ниже, чем у градиентного бустинга (0.652 против 0.69 – 0.70 с весами);
- **AdaBoost.** В обоих режимах самый низкий recall среди ансамблей (0.543 и 0.627), поэтому для задачи с приоритетом recall наименее подходящий;
- **XGBoost и CatBoost.** Наилучший баланс: с весами классов recall около 0.70 и 0.69, precision около 0.62 – 0.63, F1 около 0.66 (наибольшие значения), ROC-AUC около 0.82. Между собой статистически не различаются.

5.3. Выбор алгоритма для медицинской задачи

В задаче выявления диабета пропуск больного (FN) опаснее ложной тревоги (FP), так как ложно положительный результат устраняется дополнительным обследованием, а пропущенный диабет остаётся без лечения. Поэтому основной метрикой выбран recall, а precision и ROC-AUC используются, чтобы не допустить «накрутки» recall простым увеличением числа положительных предсказаний.

Предпочтительным алгоритмом считаем градиентный бустинг (XGBoost или CatBoost) с учётом дисбаланса классов: он находит около 70 % больных при precision около 0.63, имеет высокий ROC-AUC (≈ 0.82) и устойчивее одиночного дерева. Различие между XGBoost и

CatBoost в пределах случайных колебаний. Одиночное дерево даёт большой recall (0.747), но ценой самой низкой precision и ROC-AUC и наибольшей нестабильности. Если требуется ещё более высокий recall, надёжнее снижать порог классификации, чем менять алгоритм.

5.4. Ограничения

- тестовая выборка мала (231 объект, 81 больной); оценки recall имеют заметный разброс;
- гиперпараметры не подбирались, использованы разумные фиксированные значения;
- нулевые значения в ряде признаков (фактически пропуски) не обрабатывались; их заполнение и подбор гиперпараметров по кросс-валидации могут улучшить результаты;
- числа для XGBoost могут слегка отличаться при запуске на другой машине или версии библиотеки (случайные подвыборки в алгоритме); выводы от этого не меняются.

Вывод: на практике сравнил работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.